# Glossary

*Book: Glossary, pp. 553–558*

The book's glossary is deliberately brief — the core idea of each term, with details in the main text. The first table covers **every** book glossary entry in plain English; the A–Z section covers every other term in these notes. The "Ch" column links to the chapter notes (section in brackets). The book's Index is not reproduced.

Chapters: 1 · 2 · 3 · 4 · 5 · 6 · 7 · 8 · 9 · 10 · 11 · 12. "Part II intro" is summarised at the start of Chapter 5, "Part III intro" at the start of Chapter 10.

> **New to a word?** Every term, acronym, product and tool used in these notes — from *API*, *JSON* and *SSD* to *MVCC*, *Raft* and *write skew* — is defined once, below or in Every other term used in these notes. Networking basics (latency, caching, load balancers, queues, TCP, DNS) are covered in depth in System Design Fundamentals.

| Term | Plain-English meaning | Ch |
|---|---|---|
| **asynchronous** | Not waiting for something (e.g. a network message) to finish, and assuming nothing about how long it takes. | 5 (sync vs async replication), 8 (sync vs async networks; system models) |
| **atomic** | (1) Concurrency: an operation appears to happen at a single instant, so no one sees it half-done (cf. *isolation*). (2) Transactions: a group of writes is committed or rolled back as a whole, even under faults. | 7 (Atomicity), 9 (atomic commit, 2PC) |
| **backpressure** | Forcing a producer to slow down because the receiver can't keep up; also called *flow control*. | 11 (Messaging Systems) |
| **batch process** | A job that reads a fixed, usually large, input and writes a new dataset, leaving the input unchanged. | 10 |
| **bounded** | Having a known upper limit or size — said of network delays and datasets. | 8 (timeouts, unbounded delays), 11 (intro) |
| **Byzantine fault** | A node misbehaving arbitrarily — e.g. sending contradictory or malicious messages. | 8 (Byzantine Faults) |
| **cache** | Keeps recently used data for faster repeat reads; usually partial, so misses fall back to a slower complete store. | General term; e.g. 1, 12 |
| **CAP theorem** | A widely misunderstood theoretical result that, in the book's view, isn't useful in practice. | 9 (The CAP theorem) |
| **causality** | The dependency when one event "happens before" another: a later event answers, builds on, or only makes sense given an earlier one. | 5 (happens-before), 9 (Ordering and Causality) |
| **consensus** | Getting several nodes to agree on something (e.g. which node leads); much harder than it looks. | 9 (Fault-Tolerant Consensus) |
| **data warehouse** | A database combining data from several OLTP systems, prepared for analytics. | 3 (Data Warehousing) |
| **declarative** | Stating the properties the result should have, not the steps to produce it; for queries, an optimizer picks how to run it. | 2 (Query Languages for Data), 10 |
| **denormalize** | Deliberately adding redundancy to normalized data — usually a cache or index — to speed reads; effectively a precomputed query result, like a materialized view. | 7 (single/multi-object operations), 11 (views from one event log) |
| **derived data** | A dataset produced from other data by a repeatable process, usually to speed up particular reads (indexes, caches, materialized views). | Part III intro — 10; 12 |
| **deterministic** | Same input → same output, so it can't use random numbers, the current time, network calls or other unpredictable inputs. | General; 10, 11, 12 |
| **distributed** | Running on several networked nodes; hallmark is partial failure — some parts broken, others working, and the software often can't tell what broke. | 8 (Faults and Partial Failures) |
| **durable** | Stored so that you trust it will survive various faults. | 7 (Durability) |
| **ETL** | Extract–Transform–Load: pull data from a source database, reshape it for analytics, load it into a warehouse or batch system. | 3 (Data Warehousing) |
| **failover** | In single-leader systems, moving the leader role to another node. | 5 (Handling Node Outages) |

| Term | Plain-English meaning | Ch |
|---|---|---|
| **fault-tolerant** | Recovers automatically when something breaks (machine crash, network link down). | 1 (Reliability) |
| **flow control** | Same as *backpressure*. | 11 |
| **follower** | A replica that accepts no client writes, only changes from the leader. Aka secondary, slave, read replica, hot standby. | 5 (Leaders and Followers) |
| **full-text search** | Searching text for arbitrary keywords, often with similar-spelling or synonym matching; served by a full-text index (a kind of secondary index). | 3 (full-text search, fuzzy indexes) |
| **graph** | Vertices (referable things; aka nodes, entities) connected by edges (aka relationships, arcs). | 2 (Graph-Like Data Models) |
| **hash** | A function turning input into a random-looking number; same input → same number; different inputs rarely collide (a *collision*). Used to spread keys evenly. | 6 (Partitioning by Hash of Key) |
| **idempotent** | Safe to retry: doing it several times has the same effect as once (*idempotence*). | 11 (Idempotence), 12 |
| **index** | A data structure for quickly finding all records with a given value in a given field. | 3 (Data Structures That Power Your Database) |
| **isolation** | How much concurrent transactions can interfere with each other; serializable is strongest, weaker levels are common. | 7 (Isolation) |
| **join** | Combining records that share something — usually fetching a record referenced by another (foreign key, document reference, graph edge). | 2 (many-to-one/many relationships), 10 (reduce-side joins) |
| **leader** | The one replica allowed to make changes; chosen by election or by an administrator. Aka primary, master. | 5 (Leaders and Followers) |
| **linearizable** | Behaving as if there were one copy of the data, changed only by atomic operations. | 9 (Linearizability) |
| **locality** | Storing data together that is often needed together, for performance. | 2 (data locality for queries) |
| **lock** | Ensures only one thread, node or transaction uses something at a time; others wait for release. | 7 (Two-Phase Locking), 8 (the leader and the lock) |
| **log** | An append-only file for storing data: write-ahead log (crash safety), log-structured storage engine, replication log (leader → followers), event log (a stream). | 3 (B-tree reliability; SSTables/LSM-Trees), 5, 11 (Partitioned Logs) |
| **materialize** | Compute eagerly and store the result rather than computing on request. | 3 (data cubes, materialized views), 10 (intermediate state) |
| **node** | A running instance of software on a computer that works with other nodes over a network. | General; Part II (5–9) |
| **normalized** | No redundancy: each fact is stored and updated in exactly one place. | 2 (many-to-one/many relationships) |
| **OLAP** | Online analytic processing: workloads dominated by aggregates (count, sum, average) over many records. | 3 (Transaction Processing or Analytics?) |
| **OLTP** | Online transaction processing: fast queries reading or writing a few records, usually by key. | 3 (Transaction Processing or Analytics?) |
| **partitioning** | Splitting a dataset or computation too big for one machine across many; aka sharding. | 6 |

| Term | Plain-English meaning | Ch |
|---|---|---|
| **percentile** | Describes a distribution by counting values above/below a threshold: the 95th percentile response time *t* means 95% of requests were faster than *t*, 5% slower. | 1 (Describing Performance) |
| **primary key** | A value (number or string) uniquely identifying a record, often system-generated (sequential or random). Contrast *secondary index*. | General; see 3 |
| **quorum** | Minimum number of nodes that must vote for an operation to succeed. | 5 (quorums for reading and writing) |
| **rebalance** | Moving data or services between nodes to even out load. | 6 (Rebalancing Partitions) |
| **replication** | Keeping copies (replicas) of data on several nodes so it stays reachable if one isn't. | 5 |
| **schema** | A description of data's structure — fields and types. Data may be checked against it at various points, and it may change over time. | 2 (schema flexibility), 4 |
| **secondary index** | An extra structure beside the main data for efficiently finding records matching a condition. | 3 (Other Indexing Structures), 6 (Partitioning and Secondary Indexes) |
| **serializable** | Concurrent transactions produce the same result as running one after another in some order. | 7 (Serializability) |
| **shared-nothing** | Independent nodes, each with its own CPU, memory and disks, linked by an ordinary network (vs shared-memory or shared-disk). | Part II intro — 5 |
| **skew** | (1) Uneven load across partitions — *hot spots*. (2) A timing anomaly making events appear out of order — read skew, write skew, clock skew. | (1) 6 (hot spots), 10 (handling skew); (2) 7 (read/write skew), 8 (clock skew) |
| **split brain** | Two nodes simultaneously believing they are leader, which can break guarantees. | 5 (Handling Node Outages), 8 (truth defined by majority) |
| **stored procedure** | A transaction's logic packaged to run entirely on the database server, with no client round-trips during the transaction. | 7 (Actual Serial Execution) |
| **stream process** | A never-ending computation reading an endless event stream and producing output. | 11 |
| **synchronous** | Opposite of *asynchronous*: waiting for completion. | 5, 8 |
| **system of record** | Holds the authoritative ("source of truth") version of data; changes land here first and other datasets derive from it. | Part III intro — 10; 12 |
| **timeout** | Simplest fault detector: no response within some time. Can't tell a failed node from a network problem. | 8 (Timeouts and Unbounded Delays) |
| **total order** | An ordering where any two items (e.g. timestamps) can be compared. If some pairs can't, it's a *partial order*. | 9 (causal order is not total) |
| **transaction** | Grouping reads and writes into one logical unit to simplify error handling and concurrency. | 7 |
| **two-phase commit (2PC)** | Algorithm ensuring several database nodes all commit or all abort a transaction. | 9 (Atomic Commit and Two-Phase Commit) |
| **two-phase locking (2PL)** | Serializable isolation by locking all data a transaction reads or writes until it finishes. | 7 (Two-Phase Locking) |
| **unbounded** | Having no known limit or size; opposite of *bounded*. | 8, 11 |

## Every other term used in these notes (A–Z)

Every other technical term, acronym, algorithm, product and tool in the Preface and Chapters 1–12, alphabetically, each defined once with links to where it is used most. Terms in the table above (e.g. *quorum*, *two-phase commit*) aren't repeated; synonyms point to the one definition.

**0–9**

| Term | Plain-English meaning | Ch |
|---|---|---|
| **2PC** | See *two-phase commit* (table above). | 9 |
| **3PC (three-phase commit)** | Variant of two-phase commit designed to avoid blocking, but only correct if the network has bounded delays and failures can be detected perfectly — assumptions real networks don't meet. | 9 |

## A

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Abbreviated keys** | Storing only enough of each key in interior B-tree pages to tell ranges apart, so more keys fit per page. | 3 |
| **Abort / rollback** | Cancelling a transaction so that none of its writes take effect. | 7, 9 |
| **Abstraction** | Simpler interface that hides complicated details underneath, like SQL hiding how data is laid out on disk. | 1, 12 |
| **A/B test** | Showing two versions of a feature to different groups of users and comparing which one performs better. | 10, 12 |
| **Access path** | The route a query takes to reach the data (which indexes, which order of lookups); chosen by hand in old databases, by the query optimizer in relational ones. | 2 |
| **Accidental complexity** | Complexity caused by how something was built rather than by the problem itself; good abstractions remove it. | 1 |
| **ACID** | The classic transaction guarantees: Atomicity, Consistency, Isolation, Durability. | 7 |
| **Acknowledgment (ack)** | Message from a consumer telling the broker "I have processed this", so the broker can delete it instead of redelivering it. | 9, 11 |
| **ActiveMQ / RabbitMQ / HornetQ** | Traditional open-source message brokers (JMS/AMQP style) that delete messages once consumers acknowledge them. | 4, 11 |
| **Activity events / clickstream** | Logs of what users did (pages viewed, buttons clicked), often joined with user data in batch jobs. | 10 |
| **Actor model** | Concurrency model where independent "actors" each keep private state and communicate only by sending each other asynchronous messages. | 4 |
| **Actual serial execution** | Running transactions literally one at a time on a single thread, which is simple and serializable if each transaction is short. | 7 |
| **Admission control** | Limiting how much traffic is let into a network or system so that what is admitted gets guaranteed capacity. | 8 |
| **Aggregation** | Combining many values into one summary, such as a count, sum, average or maximum. | 3, 10 |
| **Aggregation pipeline** | MongoDB's JSON-based query language that passes documents through a series of stages (filter, group, sort...). | 2 |
| **Agile / DevOps** | Agile: working in short iterations and adapting to change. DevOps: developers and operations working together, with heavy automation. | 10 |
| **Ajax** | Technique where browser JavaScript calls the server in the background to update part of a page without reloading it. | 4 |
| **Akka** | Framework for the actor model on the Java virtual machine, which can run actors across several machines. | 4, 8 |
| **Algorithmic prison** | Being systematically shut out of jobs, loans, travel and so on because an algorithm labelled you risky. | 12 |
| **Amazon EBS** | Amazon's network-attached virtual disks for EC2 machines; reads and writes go over the network. | 8 |
| **Amazon S3** | Amazon's cloud object storage service for files of any size. | 10, 12 |
| **Amazon Web Services (AWS)** | Amazon's cloud computing platform (EC2 virtual machines, S3 storage, EBS disks and more). | Preface, 1 |

| Term | Plain-English meaning | Ch |
|---|---|---|
| **AMQP** | Advanced Message Queuing Protocol: an open standard for talking to message brokers such as RabbitMQ. | 11 |
| **Anti-caching** | In-memory database design that moves rarely used records out to disk and back when needed, so the dataset can exceed memory. | 3, 7 |
| **Anti-entropy** | Background process that compares replicas and copies over any data that one of them is missing. | 5 |
| **Apache Beam / Google Cloud Dataflow** | Beam is a programming model for batch and stream jobs that can run on several engines; Cloud Dataflow is Google's hosted engine for it. | 12 |
| **Apache BookKeeper** | Replicated log storage service built on ZooKeeper. | 9 |
| **Apache Curator** | Library of ready-made recipes (leader election, locks) on top of ZooKeeper. | 9 |
| **Apache Jena** | Open-source Java framework for RDF data and SPARQL queries. | 2 |
| **API** | Application programming interface: the operations one piece of software offers for other software to call. | 4, 9 |
| **API key** | Secret token a client sends with each request to identify and authorise itself. | 4 |
| **API versioning** | Running several versions of a service interface side by side (e.g., /v1 and /v2) so old clients keep working. | 4 |
| **Append-only** | Data that is only ever added to the end, never changed in place. | 3, 11 |
| **Archival storage** | Long-term copies of data (backups, warehouse dumps) usually written with the latest schema in one go. | 4 |
| **ASCII** | The basic 7-bit character set for English letters, digits and punctuation. | 4, 10 |
| **ASN.1 / DER** | Old, very complex standard for defining data structures and encoding them in binary; still used for TLS/SSL certificates. | 4 |
| **Asymmetric fault** | Failure where a node can receive messages but its outgoing messages are lost or delayed, so others wrongly think it's dead. | 8 |
| **Asynchronous packet network** | Network (like the internet) that gives no guarantee when, or whether, a packet will arrive. | 8 |
| **ATM (Asynchronous Transfer Mode)** | Older networking technology that reserved bandwidth like telephone circuits. | 8 |
| **Atomic (caesium) clock** | Extremely precise clock based on atomic vibrations; used with GPS in Google's TrueTime. | 8 |
| **Atomic commit** | Ensuring a transaction spread across several nodes either commits on all of them or aborts on all of them. | 9, 12 |
| **Atomicity (ACID)** | All of a transaction's writes happen, or none do; on failure, partial writes are undone. | 7, 9 |
| **Atomic write operation** | Database operation, like UPDATE x = x + 1, that reads and changes a value in one step so concurrent updates aren't lost. | 7 |
| **Auditing** | Actively checking that data is still correct and complete, rather than just trusting the system. | 12 |
| **Authentication / access control** | Checking who someone is (authentication) and what they are allowed to do (access control). | 4, 12 |
| **Availability** | The fraction of time a system is up and answering requests. | 5, 9 |
| **Avro** | Binary encoding format from Hadoop that uses a schema; reader and writer schemas can differ and are matched by field name. | 4 |

| Term | Plain-English meaning | Ch |
| --- | --- | --- |
| **Azure** | Microsoft's cloud computing platform. | 5, 11 |

## B

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Backup** | Copy of the data kept so it can be restored after loss or corruption. | 1, 7 |
| **Backward compatibility** | Newer code can read data written by older code. | 4 |
| **BASE** | Basically Available, Soft state, Eventual consistency: a loose label for "not ACID". | 7 |
| **Base64** | Way of writing binary data using only 64 printable text characters, at about 33% size overhead. | 4 |
| **Bash / shell** | Bash is the most common Unix shell: the command-line program that runs commands and pipes them together. | 3, 10 |
| **Big ball of mud** | System so tangled that it's hard to understand or change. | 1 |
| **Big Data** | Loose label for datasets too large or fast-moving for one ordinary machine. | Preface, 12 |
| **Big-O notation (O(n), O(log n))** | Way to say how cost grows with data size: O(n) grows in step with n; O(log n) grows very slowly. | 3 |
| **BigQuery / Dremel** | Google's columnar analytic query engine (Dremel) and the cloud service built on it (BigQuery). | 3 |
| **Bigtable** | Google's distributed wide-column storage system, which inspired HBase and Cassandra's data model. | 3, 6 |
| **Binary encoding** | Storing data in compact bytes rather than human-readable text. | 4 |
| **Binary string** | Sequence of raw bytes (not necessarily text). | 4 |
| **Bitcask** | Riak's default storage engine: an append-only log plus an in-memory hash map from each key to its file offset. | 3 |
| **Bitcoin / blockchain** | Cryptocurrency whose replicas keep a shared, append-only chain of blocks of transactions secured with hashes and proof of work. | 8, 12 |
| **Bitmap encoding** | In column storage, representing each distinct value as a string of bits (1 = this row has the value), which compresses very well. | 3 |
| **Blocking atomic commit** | The flaw of 2PC: if the coordinator crashes, participants may have to wait indefinitely for its decision. | 9 |
| **Bloom filter** | Small memory structure that can say "definitely not present" or "maybe present", used to skip pointless disk lookups. | 3, 11 |
| **Bottled Water / Maxwell / Debezium** | Open-source CDC tools that read a database's replication log (PostgreSQL, MySQL) and publish the changes to Kafka. | 11 |
| **Bounded / unbounded delay** | Bounded: every message arrives within a known maximum time (not provided by ordinary networks). Unbounded: no upper limit, so a timeout can never be sure a node is dead. | 8 |
| **Branching factor** | How many child pages each page of a B-tree points to (often several hundred). | 3 |
| **Branch misprediction** | When the CPU guesses wrongly which way an if-statement will go and has to discard work; vectorized code avoids many. | 3 |
| **Broadcast hash join** | Map-side join where the smaller dataset is loaded into an in-memory hash table on every mapper. | 10 |
| **Brokerless messaging** | Producers sending messages straight to consumers with no intermediary server (e.g., ZeroMQ, UDP multicast). | 11 |
| **BSON** | MongoDB's binary encoding of JSON documents. | 2, 4 |

| Term | Plain-English meaning | Ch |
|---|---|---|
| **B-tree** | The most common index structure: a tree of fixed-size pages kept sorted by key, updated in place on disk. | 3 |
| **B+ tree** | A B-tree variant that keeps all values in the leaf pages; most "B-trees" in practice are B+ trees. | 3 |
| **Bulk loading** | Building a database's files directly in a batch job and swapping them in at once, instead of writing records one by one. | 2, 10 |
| **Bulk synchronous parallel (BSP) / Pregel** | Model for graph processing in rounds: each vertex processes its incoming messages, then sends messages along edges for the next round. | 10 |
| **Bursty traffic** | Network traffic that comes in sudden spikes rather than at an even rate. | 8 |
| **Business intelligence (BI)** | Using data analysis to support business decisions, typically via reports and dashboards over a data warehouse. | 3, 10 |
| **Byte offset** | The position of a record in a file, counted in bytes from the start. | 3 |
| **Byzantine fault-tolerant** | Still working correctly even if some nodes misbehave arbitrarily or maliciously. | 8 |
| **Byzantine Generals Problem** | The classic puzzle of reaching agreement when some participants may be traitors sending conflicting messages. | 8 |

## C

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Cache hit / cache miss** | Hit means the requested data was found in the cache; a miss means it had to be fetched from the slower store. | 1 |
| **Cache invalidation** | Removing or updating cached copies when the underlying data changes so stale data isn't served. | 1, 7 |
| **Capacity planning** | Estimating future load and making sure enough hardware is available in time. | 1 |
| **Cascading** | A Java library that builds MapReduce workflows from higher-level operators such as joins and groupings. | 10 |
| **Cascading failure** | One component's failure overloads others, which then fail too, and so on. | 6, 8 |
| **Cassandra** | Open-source, Dynamo-style leaderless database with a wide-column data model, designed for many writes. | 3, 6 |
| **Catch-up recovery** | Restarted follower reading the leader's log from where it stopped until it has caught up. | 5 |
| **Caught up** | Follower that has applied every change the leader has made so far. | 5, 11 |
| **Causal context** | Riak's name for the version vector a client sends back with a write so the database can tell what it already saw. | 5 |
| **Causal dependency** | Event B depends on event A (B happened because of, or after seeing, A), so B must not be processed as if A hadn't happened. | 5, 9 |
| **Causally consistent** | Consistency model guaranteeing everyone sees cause-and-effect events in the right order; concurrent events may appear in any order. | 9 |
| **CDC (change data capture)** | Reading a database's stream of changes so other systems can apply exactly the same changes. | 11, 12 |
| **CDN (content delivery network)** | Worldwide network of cache servers that serves static content from a location near each user. | 6 |
| **CEP engines (Esper, StreamBase, SQLstream, Apama)** | Products that run continuous pattern-matching queries over event streams. | 11 |
| **Certificate transparency** | Public, Merkle-tree-based log used to check that TLS/SSL certificates were issued legitimately. | 12 |
| **Chain replication** | Replication where writes flow along a chain of nodes and reads are served by the tail, giving strong consistency. | 5, 9 |
| **Changelog** | Log of every change made to some state; replaying it rebuilds the state. | 11 |
| **Change stream / change feed** | An API that lets clients subscribe to every change in a database as it happens. | 11, 12 |
| **Chaos Monkey** | A Netflix tool that deliberately kills machines in production to make sure the system tolerates faults. | 1, 8 |
| **Checkpoint** | Saved snapshot of a computation's state and progress so it can resume from there after a crash instead of restarting. | 10, 11 |
| **Checksum** | Short value computed from data; if the data changes or is corrupted, the checksum no longer matches. | 3, 12 |

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Chubby** | Google's internal lock and coordination service, which inspired ZooKeeper. | 9 |
| **Circuit switching** | Reserving a fixed amount of network bandwidth for a whole call, as old telephone networks did; gives bounded delay. | 8 |
| **Client / server** | Client sends requests; a server answers them. A server may itself be a client of other services. | 4, 5 |
| **Clock drift** | Clock running slightly faster or slower than real time, so it slowly wanders off. | 8 |
| **Clock skew** | The difference between what two machines' clocks say at the same moment. | 5, 8 |
| **Clos topology** | Network layout with many redundant paths between switches, common in datacenters. | 8 |
| **Cloud computing** | Renting computing resources (machines, storage, managed services) over the internet on demand. | 8, 11 |
| **Cluster** | Group of machines working together as one system. | 6, 9 |
| **Clustered index** | Index that stores the whole row inside the index itself, rather than pointing elsewhere. | 3 |
| **COBOL** | Old business programming language, used with early hierarchical and network databases. | 2 |
| **CODASYL / network model** | Old (1970s) database model where records link to each other in a graph that programs navigate by following pointers. | 2 |
| **Code generation** | Producing source code classes automatically from a schema, as Thrift and Protocol Buffers do. | 4 |
| **Column-oriented storage** | Storing all values of each column together instead of each row together; fast for analytics that read a few columns of many rows. | 3 |
| **Command** | In event sourcing, a user's request that may still be rejected; once validated it becomes an immutable event (a fact). | 11 |
| **Commit** | Finishing a transaction so that all its writes become permanent and visible. | 7, 9 |
| **Commit point** | The moment in 2PC when the coordinator writes its commit decision to disk; after that, the transaction must commit. | 9 |
| **Commodity machines** | Ordinary, inexpensive servers, as opposed to specialised supercomputer hardware. | 8, 10 |
| **Commutative operation** | Operation where order doesn't matter (adding 3 then 5 equals adding 5 then 3), making conflicts easy to merge. | 7 |
| **Compaction** | Throwing away overwritten or deleted values in log files, keeping only the latest value per key. | 3, 11 |
| **Compare-and-set (CAS)** | Atomic operation: "set this value to X only if it is currently Y", else fail. | 7, 9 |
| **Compensating transaction** | Later action that corrects an earlier mistake (refund, apology, rebooking) instead of preventing it up front. | 9, 12 |
| **Complex event processing (CEP)** | Searching a stream for patterns of events described by queries that run continuously. | 11 |
| **Compound primary key** | Key made of several columns, e.g., (user_id, timestamp): the first part picks the partition, the rest sorts within it. | 6 |

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Compute-intensive application** | Application whose bottleneck is raw CPU work rather than data. | Preface, 1 |
| **Concatenated index** | Index on several fields combined in a fixed order, like a phone book sorted by (last name, first name). | 3, 6 |
| **Concurrency** | Several operations being in progress at overlapping times. | 5, 7 |
| **Concurrency control** | Mechanisms (locks, versioning, validation) that stop concurrent transactions from corrupting data. | 7, 11 |
| **Concurrent operations** | Two operations where neither knew about the other when it started — neither "happened before" the other. | 5, 9 |
| **Confidence interval (clocks)** | Range of time a clock reading is known to fall within, such as Spanner's TrueTime [earliest, latest]. | 8 |
| **Conflict avoidance** | Routing all writes for a given record to the same leader so write conflicts can't happen. | 5 |
| **Conflict resolution** | Deciding what the final value should be when two concurrent writes changed the same data. | 5, 7 |
| **Consensus properties (uniform agreement, integrity, validity, termination)** | The rules consensus must satisfy: no two nodes decide differently, nobody decides twice, the decision was proposed by someone, and every live node eventually decides. | 9 |
| **Consistency (ACID)** | The application's rules (invariants) about the data hold after every transaction; really a property of the application. | 7 |
| **Consistency model** | Precise statement of which results reads may return when there are concurrent writes and replicas. | 9 |
| **Consistent hashing** | Way of mapping keys to nodes so that adding or removing a node moves only a small share of keys. | 6 |
| **Consistent prefix reads** | Guarantee that if writes happened in some order, readers see them in that same order (never an answer before its question). | 5 |
| **Consul** | HashiCorp's tool for service discovery and configuration, built on Raft. | 9, 11 |
| **Consumer** | Program that reads and processes messages or events from a stream or queue. | 11 |
| **Consumer group** | Set of consumers sharing the work of reading a topic, each taking some partitions. | 11 |
| **Consumer offset** | The position in a log partition up to which a consumer has processed messages. | 11 |
| **Contention** | Many transactions or threads competing for the same data or lock at once. | 7 |
| **Context switch** | The CPU switching from running one thread to another, saving and restoring their state. | 1, 8 |
| **Controller node** | Node responsible for deciding which node is leader (or owns which partition). | 5, 9 |
| **Convergent conflict resolution** | Every replica resolves a conflict the same way, so all end up with the same final value. | 5 |
| **Coordination-avoiding data system** | System that keeps data integrity without synchronous cross-node coordination. | 12 |
| **Coordinator node (leaderless)** | In leaderless replication, the node that receives a client's request and forwards it to the replicas (without enforcing an | 5 |

| Term | Plain-English meaning | Ch |
|---|---|---|
| | order). | |
| **Coordinator (transaction manager)** | The component that runs two-phase commit, asking participants to prepare and then telling them to commit or abort. | 9 |
| **Copy-on-write** | Instead of changing data in place, write a modified copy and switch pointers to it, leaving the old version intact. | 3, 7 |
| **CORBA / DCOM / RMI / EJB** | Older RPC frameworks for calling objects on remote machines; mostly abandoned because they hid network problems. | 4 |
| **Cost-based query optimizer** | Optimizer that estimates the cost of each possible execution plan and picks the cheapest. | 10 |
| **Couchbase** | Distributed document/key-value database descended from Memcached and CouchDB. | 6 |
| **CouchDB** | Open-source document database that stores JSON and supports multi-leader replication. | 2, 5 |
| **Covering index** | Index that stores some extra columns so certain queries can be answered from the index alone. | 3 |
| **CPU** | Central processing unit: the chip that executes program instructions. | 1, 8 |
| **CQRS** | Command Query Responsibility Segregation: keeping the write model separate from (possibly many) read models. | 11 |
| **Crash-recovery fault** | Fault model where nodes may crash and later restart, keeping what was on disk. | 8 |
| **Crash-stop fault** | Fault model where a crashed node is gone forever. | 8, 9 |
| **CRDT** | Conflict-free replicated data type: a data structure (counter, set, list...) that merges concurrent edits automatically and sensibly. | 5 |
| **Cross-device read-after-write** | Read-your-writes across a user's devices: an update made on a laptop must be visible on their phone. | 5 |
| **Cross-partition transaction** | Transaction touching data on several partitions, which needs coordination between them. | 7 |
| **C-Store** | Research column-store database (the ancestor of Vertica) that kept data sorted in several orders. | 3 |
| **CSV** | Comma-separated values: a simple text format for tables, ambiguous about types and escaping. | 3, 4 |
| **curl** | Command-line tool for making HTTP requests. | 4, 10 |
| **Cursor stability** | Locking a row while it is being read and modified, to prevent lost updates. | 7 |
| **Cypher** | The declarative query language of the Neo4j graph database. | 2 |

## D

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Daemon** | Background program that runs continuously on a server (e.g., the HDFS process on each machine). | 6, 10 |
| **DAG (directed acyclic graph)** | Graph of steps with arrows and no cycles; dataflow engines run jobs as DAGs of operators. | 10 |
| **Database inside-out** | Another name for the unbundling / dataflow approach to building systems from logs and derived views. | 12 |
| **Database-internal distributed transaction** | Distributed transaction where all participants run the same database software, so it can be optimised. | 9, 12 |
| **Data broker** | Company that buys, combines, infers and resells personal data. | 12 |
| **Databus / Wormhole / Sherpa** | CDC systems built at LinkedIn (Databus), Facebook (Wormhole) and Yahoo! (Sherpa). | 11 |
| **Datacenter** | Building full of servers, networking and power equipment. | 5, 8 |
| **Data cube / OLAP cube** | Precomputed grid of aggregates over several dimensions, so totals can be read instantly. | 3 |
| **Data exhaust** | Label for behavioural data produced as a side effect of using a service. | 12 |
| **Dataflow** | The path data takes as it is passed and transformed between processes or systems. | 10, 12 |
| **Dataflow engine** | Batch processor like Spark, Tez or Flink that runs a whole workflow as one job without writing every intermediate step to disk. | 10 |
| **Data integration** | Making sure the same data ends up, in the right shape, in every system that needs it. | 12 |
| **Data-intensive application** | Application whose main challenge is the amount, complexity or speed of change of its data, not CPU power. | Preface, 1 |
| **Data lake** | Big store (often HDFS or object storage) where raw data of any format is dumped first and interpreted later. | 10 |
| **Data locality** | Running computation on the machine that already stores the input, to avoid copying it over the network. | 2, 10 |
| **Datalog** | Old logic query language (a subset of Prolog) that builds queries from small rules. | 2 |
| **Data model** | How data is organised and described to the programmer: tables, documents, graphs and so on. | 2 |
| **Data outlives code** | Code is replaced in minutes, but stored data may stay years old, so new code must keep reading old data. | 4 |
| **Data system** | The book's umbrella term for databases, caches, queues and other tools that store and process data. | 1, 12 |
| **Datomic** | Database that keeps all history as immutable facts and supports Datalog queries. | 2, 7 |
| **DB2** | IBM's commercial relational database. | 2, 7 |
| **Deadlock** | Two transactions each waiting for a lock the other holds, so neither can proceed; the database aborts one. | 7 |
| **Declarative query language** | A *declarative* (see table above) language such as SQL. | 2, 10 |

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Decoding (deserialization, parsing)** | Turning bytes back into in-memory objects. | 4 |
| **Denial of service** | Attack (or accident) that floods a system so it can't serve legitimate users. | 8 |
| **Derivation function** | The code that turns source data into a derived dataset (e.g., "extract the field and sort it" for an index). | 12 |
| **Dimension table** | In a star schema, a table describing the who/what/where of events (products, stores, dates). | 3 |
| **Dirty read** | Seeing another transaction's writes before it has committed. | 7 |
| **Dirty write** | Overwriting another transaction's uncommitted write. | 7 |
| **Distributed actor framework** | System (Akka, Orleans, Erlang/OTP) that runs actors across several machines with the same message-passing model. | 4, 9 |
| **Distributed filesystem** | Filesystem whose files are spread across many machines (e.g., HDFS). | 10 |
| **Distributed RPC** | Storm's feature for sending a query through a stream topology and gathering results from many partitions. | 11, 12 |
| **Django / ActiveRecord / Hibernate** | Popular ORM frameworks for Python (Django), Ruby on Rails (ActiveRecord) and Java (Hibernate). | 2, 7 |
| **DNS** | Domain Name System: translates names like example.com into IP addresses. | 6, 9 |
| **Docker / Mesos** | Tools for packaging programs into containers (Docker) and scheduling them across a cluster (Mesos). | 10, 12 |
| **Document database** | Database storing self-contained records as JSON-like documents (e.g., MongoDB). | 2 |
| **Document-partitioned (local) index** | Each partition keeps a secondary index of only its own records; reads must ask all partitions. | 6 |
| **Document reference** | Field in one document holding the ID of another, the document-model equivalent of a foreign key. | 2, 10 |
| **DOM** | Document Object Model: the browser's tree of page elements that JavaScript can change. | 2 |
| **Domain-driven design (DDD)** | Software design approach centred on modelling the business domain; event sourcing is popular in it. | 11 |
| **Dotted version vector** | Refined version vector used by Riak to track causality precisely. | 5 |
| **Double-spending** | Spending the same money or credit twice due to a concurrency bug. | 7 |
| **Drill-down / slicing and dicing** | Exploring analytic data by zooming into detail or viewing it by different dimensions. | 3 |
| **Druid** | Open-source analytic database designed for fast queries over event data. | 11 |
| **Dryad / Nephele** | Research dataflow engines that inspired Spark, Tez and Flink. | 10 |
| **Dual writes** | Application code writing the same change to two systems itself; it can leave them permanently inconsistent. | 11, 12 |
| **Duration** | Amount of elapsed time (e.g., how long a request took), as opposed to a point in time. | 8 |
| **Dynamic membership** | Adding or removing nodes from a consensus group while it keeps running. | 9 |

| Term | Plain-English meaning | Ch |
| --- | --- | --- |
| **Dynamic partitioning** | Splitting a partition when it grows too big and merging small ones, so the number of partitions follows data volume. | 6 |
| **Dynamo** | Amazon's in-house leaderless key-value store, whose paper inspired Riak, Cassandra and Voldemort ("Dynamo-style"). | 5, 9 |
| **DynamoDB** | Amazon's hosted cloud key-value/document database (different from the original Dynamo design). | 5, 6 |

## E

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Edge** | Connection between two vertices in a graph. | 2 |
| **Edit distance** | How many single-letter insertions, deletions or substitutions turn one word into another; used for fuzzy search. | 3 |
| **Elasticsearch** | Popular open-source distributed search engine built on Lucene. | 6, 11 |
| **Elastic system** | System that can automatically add or remove machines as load changes. | 1, 6 |
| **Elm / React / Redux** | Front-end tools that model the user interface as a function of a stream of events and state changes. | 12 |
| **Encapsulation** | Hiding a component's internal details behind a well-defined interface. | 4 |
| **Encoding (serialization, marshalling)** | Turning in-memory objects into bytes for storage or sending over a network. | 4 |
| **Encryption** | Scrambling data so only holders of the right key can read it. | 4, 12 |
| **End-to-end argument** | Some guarantees (dedup, integrity, security) can only be fully provided by the endpoints of a communication, not by lower layers alone. | 12 |
| **Ephemeral node** | A ZooKeeper entry that is deleted automatically when the client session that created it dies. | 9 |
| **Epoch number (ballot / view / term)** | Counter that increases with each leader election, so messages from an old leader can be recognised and ignored. | 9 |
| **Epoch (time)** | The fixed starting point clocks count from, usually midnight UTC on 1 January 1970. | 8 |
| **Equi-join** | Join where records are matched on equal key values. | 10 |
| **Erasure coding** | Storing data with extra computed parity pieces so lost parts can be rebuilt, using less space than full copies. | 10 |
| **Erlang / OTP** | Programming language and framework built around lightweight processes (actors) and fault tolerance. | 4 |
| **Error-correcting codes** | Extra bits stored or sent with data so that small errors can be detected and fixed automatically. | 8, 10 |
| **Espresso** | LinkedIn's distributed document database. | 4, 6 |
| **Etcd** | Open-source coordination service and key-value store using the Raft consensus algorithm (used by Kubernetes). | 9 |
| **Ethernet** | The standard technology for wired local networks. | 8 |
| **Etherpad / Google Docs** | Real-time collaborative editors where several people type into the same document at once. | 5 |
| **Event** | Small, immutable record that something happened at a certain time. | 11, 12 |
| **Event sourcing** | Storing every change as an immutable event in a log and deriving current state from those events. | 11, 12 |
| **Event time** | When an event actually happened, according to the device that produced it. | 11, 12 |
| **Eventual consistency** | If writes stop, all replicas will eventually hold the same data; until then reads may be stale. | 5, 9 |

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Evolvability** | How easily a system can be changed to meet new requirements. | 1, 4 |
| **Exactly-once (effectively-once) semantics** | The end result is as if each message were processed once, even though retries may have happened. | 11, 12 |
| **Excision / shunning** | Actually deleting data from an immutable event history, e.g., for legal reasons. | 11 |
| **Explicit locking** | The application locking rows itself (e.g., SELECT ... FOR UPDATE) to prevent lost updates. | 7 |
| **Exponential backoff** | Waiting longer and longer (e.g., doubling) between retries so an overloaded system can recover. | 7 |

## F

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Fact table** | The huge central table of a star schema, one row per event (such as each sale). | 3 |
| **Failure** | The whole system stops providing its service to users (as opposed to a fault in one part). | 1, 8 |
| **Fan-out** | One input causing many outputs: e.g., one tweet delivered to every follower's timeline, or one message sent to every consumer. | 1, 11 |
| **Fault** | One component deviating from its specification (a disk dying, a bug); a failure is when the whole system stops serving users. | 1, 8 |
| **Federated database / polystore** | One query interface that reads from many different underlying stores ("unifying reads"). | 12 |
| **Feedback loop** | Prediction changes the world in a way that reinforces the prediction. | 12 |
| **Fencing / STONITH** | Forcibly cutting off or shutting down a node that may wrongly believe it is still leader ("Shoot The Other Node In The Head"). | 5 |
| **Fencing token** | Number that increases each time a lock or lease is granted; storage rejects writes carrying an older number. | 8, 9 |
| **Fibre Channel / SAN / NAS** | Specialised shared-storage hardware: a storage area network (SAN) or network-attached storage (NAS) reached over dedicated links. | 10 |
| **Field tag** | Number that identifies a field in Thrift or Protocol Buffers encodings, instead of writing the field name. | 4 |
| **Finite state automaton** | Small machine of states and transitions that accepts or rejects strings; Lucene uses one for fuzzy search. | 3 |
| **Firebase / Meteor** | Application platforms that let clients subscribe to live database changes. | 11 |
| **Firewall** | Network filter that blocks traffic not explicitly allowed. | 4, 8 |
| **Fixed number of partitions** | Creating many more partitions than nodes up front, then moving whole partitions between nodes when rebalancing. | 6 |
| **Flink** | Apache Flink: an open-source stream processor that also does batch jobs. | 10, 11 |
| **FLP result** | Proof that consensus can't always be reached in a purely asynchronous system if a node may crash; timeouts get round it in practice. | 9 |
| **FlumeJava / Crunch** | Libraries for writing batch pipelines as chains of high-level operations that compile down to MapReduce or Spark jobs. | 10 |
| **Foreign key** | Column that refers to the primary key of a row in another table. | 2, 7 |
| **Forward compatibility** | Older code can read data written by newer code (ignoring fields it doesn't know). | 4 |
| **Forwarding pointer** | Note left in a heap file when a row moves, pointing to the row's new location. | 3 |
| **FoundationDB** | Distributed key-value store offering serializable ACID transactions. | 7 |
| **Fractal tree** | A B-tree variant that buffers writes in internal pages to reduce disk I/O. | 3 |
| **Fragmentation** | Free space scattered in small unusable gaps across disk pages. | 3 |
| **fsync** | The system call that forces data buffered in memory onto disk. | 7, 9 |
| **Full table scan** | Reading every row of a table, without using an index. | 10 |

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Functional vs nonfunctional requirements** | Functional: what the system must do. Nonfunctional: qualities like security, reliability, scalability and compliance. | 1 |
| **Futures / promises** | Objects representing a result that will be available later, used for asynchronous calls. | 4, 12 |

## G

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Garbage collection (GC)** | Automatically freeing memory (or old data versions) that is no longer used; can cause program pauses. | 7, 8 |
| **GFS (Google File System)** | Google's distributed file system, which inspired HDFS. | 10 |
| **Giraph / GraphChi / GraphX** | Graph processing frameworks implementing the Pregel model or single-machine variants. | 10 |
| **Git** | The most widely used version control system; it tracks history as branches and merges. | 5, 9 |
| **GlusterFS / QFS** | Other distributed filesystems, alternatives to HDFS. | 10 |
| **GoldenGate** | Oracle's tool for log-based change data capture and replication. | 5, 11 |
| **Google Cloud Pub/Sub** | Google's hosted publish/subscribe messaging service. | 11 |
| **Gossip protocol** | Nodes randomly exchanging state with each other so information spreads through the whole cluster. | 6 |
| **GPS** | Global Positioning System: satellites whose signals give location and very accurate time. | 8 |
| **Graph database** | Database built around vertices and edges, good for highly connected data (e.g., Neo4j). | 2 |
| **Gremlin** | Graph traversal query language (from Apache TinkerPop). | 2 |
| **gRPC** | Google's open-source RPC framework using Protocol Buffers over HTTP/2. | 4 |

## H

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Hadoop** | The open-source ecosystem (HDFS, MapReduce, YARN...) for storing and processing huge datasets on clusters. | 10 |
| **Happens-before** | Operation A happened before B if B knew about A, depended on it, or built on it. | 5, 9 |
| **Hard real-time system** | System that must respond within a strict deadline or it has failed (e.g., airbag controllers). | 8 |
| **Hash function** | See *hash* (table above). | 6 |
| **Hash index** | In-memory hash map from each key to the location of its value on disk. | 3 |
| **Hash mod N** | Choosing a partition with hash(key) % N; a bad idea because changing N moves almost every key. | 6 |
| **Hash partitioning** | Assigning records to partitions based on a hash of their key, which spreads load evenly but loses key ordering. | 6 |
| **HATEOAS** | Strict REST principle where responses include links telling the client what it can do next. | 4 |
| **HBase** | Open-source Bigtable-style wide-column database on top of HDFS. | 3, 6 |
| **HDFS** | Hadoop Distributed File System: stores huge files across many machines, replicating blocks for safety. | 10 |
| **Head-of-line blocking** | Slow item at the front of a queue holding up everything behind it. | 1, 11 |
| **Heap file** | File holding table rows in no particular order; indexes point into it. | 3 |
| **Heartbeat** | Small periodic "I'm alive" message; missing several in a row suggests the node has failed. | 9 |
| **Helix** | LinkedIn's cluster-management framework that uses ZooKeeper to track partition assignments. | 6 |
| **Heterogeneous distributed transaction** | Transaction spanning different technologies (e.g., two databases from different vendors plus a message broker). | 9, 12 |
| **Heuristic decision** | Participant in 2PC unilaterally deciding to commit or abort an in-doubt transaction, breaking atomicity. | 9 |
| **Hierarchical model / IMS** | IBM's 1960s database that stored records as a tree, each record with one parent. | 2 |
| **High-frequency trading** | Automated trading that relies on microsecond-precise timing. | 8 |
| **High-performance computing (HPC)** | Supercomputers running large scientific calculations; they usually stop the whole job on any failure. | 8 |
| **Hinted handoff** | With a sloppy quorum, a node temporarily stores writes for an unreachable node and passes them on when it returns. | 5 |
| **Hive** | A Hadoop tool that runs SQL-like queries as batch jobs over files in HDFS. | 10 |
| **Hive metastore / HCatalog** | Catalogs recording which files in HDFS form which tables and how they are partitioned. | 10 |
| **Hot key / linchpin object** | Single key with disproportionate traffic (e.g., a celebrity's account), causing skew. | 6, 10 |
| **Hot spot** | Partition with far more load than others; see *skew* (table above). | 6, 10 |

| Term | Plain-English meaning | Ch |
| --- | --- | --- |
| **HTML / CSS** | The markup language (HTML) and style language (CSS) of web pages. | 2, 4 |
| **HTTP** | The request/response protocol of the web. | 4, 12 |
| **Human fault tolerance** | Being able to recover from buggy code or mistakes, e.g., by rerunning a job over immutable input. | 10 |
| **HyperLogLog** | Probabilistic algorithm that estimates how many distinct items a stream contains, using very little memory. | 11 |
| **Hypervisor** | The software that runs virtual machines and shares the physical hardware between them. | 8 |

## I

| Term | Plain-English meaning | Ch |
|---|---|---|
| **IaaS** | Infrastructure as a service: renting virtual machines, storage and networks from a cloud provider. | Preface |
| **ICMP Destination Unreachable** | Network message a router may send when it can't deliver a packet; not always sent or reliable. | 8 |
| **ID** | Identifier: a value (number or string) that uniquely names something. | 10, 12 |
| **Idempotence** | See *idempotent* (table above). | 11, 12 |
| **IDL (interface definition language)** | Small language for describing a schema or service interface, from which code can be generated. | 4 |
| **Immutable** | Never changed after it is written; new information is added instead. | 10, 11 |
| **Impala / Presto** | Open-source MPP SQL query engines that read data stored in Hadoop. | 10 |
| **Impedance mismatch** | The awkward translation needed between objects in application code and rows in relational tables. | 2 |
| **Imperative language** | Code that spells out step by step how to compute something, in a specific order. | 2 |
| **Increment-and-get** | Atomic operation that adds one to a counter and returns the new value. | 9 |
| **Index-range lock (next-key lock)** | Simplified predicate lock that locks a whole range in an index, used to prevent phantoms. | 7 |
| **In doubt (uncertain)** | The state of a 2PC participant that voted "yes" and must wait for the coordinator's decision. | 9 |
| **InfiniBand** | Very fast network technology used in supercomputers. | 8 |
| **Initial snapshot** | Full copy of a database used to start CDC, before following the change stream. | 11, 12 |
| **In-memory database** | Database that keeps all its data in RAM (with disk or replication for durability). | 3, 4 |
| **InnoDB** | MySQL's default storage engine, which uses B-trees and supports transactions. | 3, 7 |
| **Integrity** | No lost, duplicated, contradictory or false data; violations are permanent until repaired. | 9, 12 |
| **Internet of Things** | Everyday devices (sensors, appliances, toys) connected to the internet. | 6, 12 |
| **Invariant** | Rule about the data that must always be true (e.g., credits and debits balance). | 7 |
| **Inversion of control** | Letting a framework call your code and wire inputs to outputs, rather than your code wiring everything itself. | 10 |
| **Inverted index** | Map from each word to the list of documents containing it; the core of full-text search. | 12 |
| **IP** | Internet Protocol: delivers individual packets between machines, with no guarantee of delivery or order. | 6, 8 |
| **Isolation level** | Setting choosing how strongly a database isolates concurrent transactions (read committed, snapshot isolation, serializable...). | 7, 12 |
| **Iterator** | Object that hands out items one at a time, such as the values a reducer receives for one key. | 10 |

## J

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Java** | Widely used programming language that runs on the Java Virtual Machine (JVM). | 4, 9 |
| **Java EE** | Java Enterprise Edition: a set of standards (JTA, JMS, EJB...) for business applications on Java. | 9 |
| **JavaScript** | The programming language that runs in web browsers (and on servers via Node.js). | 2, 4 |
| **Jepsen** | Kyle Kingsbury's test harness that injects faults into distributed databases to check their claimed guarantees. | 12 |
| **Jitter** | Random variation in network delay. | 8 |
| **JMS** | Java Message Service: the standard Java API for message brokers. | 11 |
| **Job scheduler** | Software that decides which machines run which tasks (e.g., YARN). | 9 |
| **JSON** | JavaScript Object Notation: a popular text format of nested objects, arrays, strings and numbers. | 2, 4 |
| **JTA / JDBC** | Java standards: JTA coordinates distributed (XA) transactions; JDBC is the standard API for connecting to SQL databases. | 4, 9 |
| **JVM** | Java Virtual Machine: the runtime that executes Java (and Scala, Kotlin...) programs; it has garbage-collection pauses. | 8, 10 |

## K

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Kafka** | Apache Kafka: a durable, partitioned, replicated log used as a message broker and event store. | 11 |
| **Kafka Connect** | Framework for streaming data between Kafka and other databases and systems. | 11 |
| **Kafka Streams** | Stream processing library built into the Kafka project. | 11 |
| **Key-range partitioning** | Giving each partition a continuous range of sorted keys, which makes range scans easy but risks hot spots. | 3, 6 |
| **Key-value store** | Database that stores values looked up by a unique key, like a big persistent hash map. | 3, 10 |
| **Kinesis** | Amazon's hosted log-based streaming service, similar to Kafka. | 11 |
| **k-nearest neighbors** | Finding the k items most similar to a given one, e.g., for recommendations. | 10 |
| **Kryo / pickle** | Language-specific serialization libraries (Kryo for Java, pickle for Python). | 4 |
| **Kubernetes** | Open-source system for deploying and running containerised applications across a cluster. | 10, 12 |

## L

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Lambda architecture** | Running a batch system and a stream system side by side over the same events, then merging their outputs. | 12 |
| **Lamport timestamp** | Pair (counter, node ID) that gives a total order of events consistent with causality, without synchronised clocks. | 9 |
| **Language-specific format** | Encoding built into one programming language (Java serialization, Python pickle); risky and not portable. | 4 |
| **Last write wins (LWW)** | Resolving conflicts by keeping the write with the latest timestamp and silently discarding the others. | 5, 7 |
| **Latch** | Lightweight lock protecting an in-memory data structure, such as a B-tree page during an update. | 3 |
| **Latency** | Time spent waiting: e.g., the delay before a request is handled, or the network delay of a message. | 1, 8 |
| **Leader-based replication** | One replica (the leader) takes all writes and streams them to the followers. | 5 |
| **Leader election** | Choosing which node is the leader, normally via a consensus algorithm. | 9 |
| **Leaderless replication** | Any replica accepts writes from clients directly; reads and writes use quorums (Dynamo-style). | 5, 9 |
| **Leaf page** | The bottom level of a B-tree, which holds the actual values or row references. | 3 |
| **Leap second** | Extra second occasionally added to keep clocks in step with Earth's rotation; it has crashed systems. | 8 |
| **Lease** | Lock that expires automatically after a time unless renewed. | 8 |
| **Ledger** | Append-only record of financial transactions, a natural fit for event sourcing. | 7, 11 |
| **LevelDB** | Google's open-source LSM-tree key-value storage library. | 3 |
| **Levenshtein automaton** | Data structure that efficiently finds words within a given edit distance of a search term. | 3 |
| **Limping node** | Node that is alive but extremely slow, which can be worse than a dead one. | 8 |
| **Linux / Unix** | Families of operating systems built around small programs, files and pipes; Linux is the most widely used. | 10 |
| **Live migration** | Moving a running virtual machine to another host, briefly pausing it. | 8 |
| **Liveness property** | "Something good eventually happens" (e.g., a request eventually gets a response). | 8, 9 |
| **LLVM** | Compiler toolkit that some query engines use to turn queries into fast machine code. | 10 |
| **LMDB / RocksDB** | Embedded key-value storage libraries: LMDB uses a copy-on-write B-tree, RocksDB an LSM-tree. | 3 |
| **Load balancer** | Component that spreads incoming requests across several servers. | 4, 6 |
| **Load balancing (messaging)** | Each message delivered to one of several consumers, to share the work. | 6, 11 |
| **Load parameters** | Numbers describing load, such as requests per second or read/write ratio. | 1 |
| **Location transparency** | Making a remote call look exactly like a local one; misleading because networks behave differently. | 4 |

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Lock-free data structure** | Concurrent data structure that uses atomic CPU instructions instead of locks. | 8 |
| **Lock (shared / exclusive)** | Shared lock lets many readers in at once; an exclusive lock admits only one holder (typically a writer). | 7 |
| **Log-based message broker** | Broker (like Kafka) that stores messages in an append-only log that consumers read by offset. | 11, 12 |
| **Log compaction** | Keeping only the latest record per key in a log, so it doesn't grow forever. | 11 |
| **Logical clock** | Counter-based clock that orders events by causality rather than by physical time. | 8, 9 |
| **Logical (row-based) log** | Replication log describing row changes independently of the storage engine's internals. | 5 |
| **Logical timestamp** | Counter-based timestamp (e.g., Lamport) that orders events without relying on physical clocks. | 12 |
| **Log segment** | One file of a log that has been split into several pieces. | 3 |
| **Log sequence number (LSN)** | The position of an entry in a database's replication log. | 5, 11 |
| **Loose coupling** | Components that know little about each other, so each can change or fail without breaking the others. | 10, 12 |
| **Lost update** | Two read-modify-write cycles run concurrently and one overwrites the other's change. | 7 |
| **LRU (least recently used)** | Eviction rule that drops the item that hasn't been used for the longest time. | 3 |
| **LSM-tree** | Log-Structured Merge-Tree: storage that buffers writes in memory, flushes sorted files, and merges them in the background. | 3 |
| **Lucene** | The open-source full-text indexing library behind Elasticsearch and Solr. | 3, 10 |

## M

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Machine learning (ML)** | Software that learns patterns from data to make predictions or decisions. | 10, 12 |
| **Mahout / MADlib** | Libraries of machine-learning algorithms that run on Hadoop (Mahout) or inside an MPP database (MADlib). | 10 |
| **Mainframe** | Large, centralised business computer of the kind that dominated before networks of commodity servers. | 2 |
| **Maintainability** | How easily people can operate, understand and change a system over its lifetime. | 1 |
| **Many-to-one / many-to-many relationship** | Many records referring to one shared record (many people living in one city), or many records relating to many others (people and the organisations they worked for). | 2 |
| **Mapper** | The function in MapReduce that runs once per input record and emits key–value pairs. | 10 |
| **MapReduce** | Batch programming model: a map function emits key–value pairs, they are sorted and grouped by key, and a reduce function processes each group. | 10 |
| **Map-side join** | Join done in the mappers, possible when inputs are small or already partitioned and sorted compatibly. | 10 |
| **Map-side merge join** | Map-side join when both inputs are partitioned and sorted the same way, so they can be merged directly. | 10 |
| **Materialized view** | Query result stored on disk and kept updated, rather than recomputed every time. | 11, 12 |
| **Materializing conflicts** | Creating rows that exist only so there is something to lock, turning a phantom into a lock conflict. | 7 |
| **Maven / npm / RubyGems** | Package managers that download libraries (dependencies) for Java, JavaScript and Ruby projects. | 10 |
| **Median** | The middle value: half of the measurements are below it, half above (the 50th percentile). | 1 |
| **Membership service** | Service (often ZooKeeper-based) that tracks which nodes are currently alive and part of the cluster. | 9 |
| **Memcached** | Simple, widely used in-memory key-value cache. | 1, 3 |
| **Memory barrier / fence** | A CPU instruction that forces memory reads and writes to be ordered and visible across cores. | 9 |
| **MemSQL** | In-memory relational database (now called SingleStore). | 3, 7 |
| **Memtable** | The in-memory sorted structure where an LSM-tree collects new writes before flushing them to disk. | 3 |
| **Mergeable persistent data structure** | Data structure that tracks history like Git and merges concurrent changes with a three-way merge. | 5 |
| **Merge (segments)** | Combining several sorted files into one, keeping the newest value per key. | 3, 5 |
| **Merkle tree** | Tree of hashes that lets you cheaply prove a record is part of a dataset. | 12 |
| **Message broker (message queue)** | Server that receives messages from producers and delivers them to consumers, buffering them in between. | 4, 11 |
| **MessagePack** | Binary encoding of JSON-like data. | 4 |

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Messaging system** | Software that delivers messages (events) from producers to consumers, directly or via a broker. | 11 |
| **Microbatching** | Processing a stream as a series of tiny batch jobs, each covering a second or so. | 11 |
| **Microservices** | Architecture where an application is split into many small services, each deployable independently. | 4, 12 |
| **Middleware** | Software sitting between applications and the operating system or network, e.g., message brokers. | 4 |
| **Migrating data** | Rewriting existing data into a new schema, which is expensive on large datasets. | 4 |
| **Minimizing irreversibility** | Designing so that mistakes can be undone, e.g., by keeping old outputs and immutable inputs. | 10 |
| **MongoDB** | Popular open-source document database storing JSON-like documents. | 2, 6 |
| **MongoDB oplog** | MongoDB's replication log, which CDC tools can read. | 11 |
| **Monotonically increasing** | Always going up (never decreasing or repeating), like a sequence of fencing tokens. | 8, 9 |
| **Monotonic clock** | Clock that only moves forward; good for measuring durations, meaningless across machines. | 8 |
| **Monotonic reads** | Guarantee that a user never sees data go backward in time across successive reads. | 5 |
| **MPP (massively parallel processing)** | Databases that split each analytic SQL query across many machines. | 10 |
| **MTTF (mean time to failure)** | The average time a component runs before it fails; with thousands of disks, some fail every day. | 1 |
| **Multi-dimensional index** | Index that can search on several columns at once, like latitude and longitude (e.g., R-trees). | 3, 10 |
| **Multi-leader replication** | More than one node accepts writes, each passing changes to the others; conflicts are possible. | 5, 9 |
| **Multi-object transaction** | Transaction that reads or writes several records together. | 7, 9 |
| **Multi-Paxos** | The version of Paxos that agrees on a sequence of values (a log) rather than a single value. | 9 |
| **Multi-tenant** | Many customers sharing the same hardware or service. | 8, 10 |
| **Mutex / semaphore** | Synchronisation tools within one machine: a mutex lets one thread at a time into a section; a semaphore lets up to N. | 7, 8 |
| **MVCC (multi-version concurrency control)** | Keeping several versions of each row so each transaction can read a consistent snapshot without blocking writers. | 7 |
| **MySQL** | Popular open-source relational database. | 5, 7 |

## N

| Term | Plain-English meaning | Ch |
|---|---|---|
| **NameNode / DataNode** | In HDFS, the NameNode tracks which machine holds each file block; DataNodes store the blocks. | 10 |
| **Neo4j** | Popular graph database that uses the Cypher query language. | 2 |
| **Network congestion** | Too much traffic for a network link, causing queues and packet loss. | 8 |
| **Network partition / netsplit** | Some nodes can't talk to others because of a network fault, though both groups are still running. | 8, 9 |
| **Noisy neighbor** | Another customer's workload on shared hardware slowing your system down. | 8 |
| **Nondeterministic** | Can give different results each time (e.g., reading the clock or a random number). | 5, 8 |
| **Nondeterministic function** | Function such as NOW() or RAND() that gives a different result each time, which breaks statement-based replication. | 5 |
| **NoSQL** | Databases that don't use the classic relational/SQL model (document, key-value, wide-column, graph stores). | Preface, 2 |
| **NTP** | Network Time Protocol: adjusts a computer's clock using time servers over the network. | 8 |
| **NUMA** | Non-uniform memory access: on big servers, memory near a CPU is faster to reach than memory near another CPU. | 5 |
| **NVM (non-volatile memory)** | Memory that keeps its contents without power while being nearly as fast as RAM. | 3 |

## O

| Term | Plain-English meaning | Ch |
|---|---|---|
| **OAuth** | Standard that lets a user grant an application limited access to their account on another service without sharing their password. | 4 |
| **Object container file** | An Avro file holding many records, with the writer's schema stored once at the start. | 4 |
| **Object database / XML database** | 1980s–2000s alternatives to relational databases that stored objects or XML documents; they faded away. | 2 |
| **Object storage** | Cloud storage of whole files ("objects") by name, such as Amazon S3. | 10, 12 |
| **Observer pattern** | Code subscribes to a value and is notified when it changes, instead of repeatedly checking. | 12 |
| **Offline-first application** | App that works mainly against a local on-device database and syncs with servers in the background. | 5, 12 |
| **Offset** | Position number: the byte position in a file, or a message's sequence number within a log partition. | 11 |
| **One-to-many relationship** | One record owning a list of others, like a résumé with many job entries; a natural fit for a tree or document. | 2 |
| **Online system (service)** | System that waits for requests and must reply quickly; response time is its key metric. | 10 |
| **OpenAPI (Swagger)** | Standard format for describing RESTful APIs. | 4 |
| **OpenStack** | Open-source software for running a private cloud. | 9, 10 |
| **Operability** | How easy it is for operations teams to keep a system running smoothly. | 1 |
| **Operational transformation** | The conflict-resolution technique behind collaborative editors like Google Docs. | 5 |
| **Operation identifier / request ID** | Unique ID generated by the client for one logical action, used to detect and drop duplicates everywhere downstream. | 12 |
| **Operator** | One processing step in a dataflow or stream job (map, filter, join...). | 10, 11 |
| **Optimistic concurrency control** | Let transactions proceed without blocking and check for conflicts at commit time, aborting if necessary. | 7 |
| **Oracle** | Oracle Database: a widely used commercial relational database. | 5, 7 |
| **Orleans / Finagle / Rest.li** | RPC and actor frameworks from Microsoft (Orleans), Twitter (Finagle) and LinkedIn (Rest.li). | 4 |
| **ORM (object-relational mapping)** | Library that maps program objects to database rows to reduce boilerplate. | 2, 7 |
| **Orphaned transaction** | In-doubt transaction whose coordinator lost its decision, so it holds locks until someone intervenes manually. | 9 |
| **OS (operating system)** | The software that manages a computer's hardware and runs programs (Linux, Windows...). | 8, 10 |
| **Overcommitted resources** | Promising more CPU or memory to tasks than physically exists, betting they won't all use it at once. | 10 |

## P

| Term | Plain-English meaning | Ch |
|---|---|---|
| **p50 / p95 / p99 / p999** | p50, p95, p99, p999 are percentiles: e.g., p99 is the response time 99% of requests beat. | 1 |
| **Packet** | Small chunk of data sent across a network on its own. | 8 |
| **Packet switching** | Networks that send data as independent packets sharing links dynamically; efficient but delays are unbounded. | 8 |
| **Page** | Fixed-size block (often 4 KB) that disks and B-trees read and write as a unit. | 3 |
| **Page cache** | Memory the operating system uses to keep recently read disk pages, making repeated reads fast. | 10 |
| **Page fault / swapping** | The OS moving memory to and from disk because RAM is full, which can pause a program unexpectedly. | 8 |
| **PageRank** | Google's algorithm ranking web pages by how many important pages link to them; a classic graph computation. | 2, 10 |
| **Parquet** | Popular open-source columnar file format for analytics. | 4, 10 |
| **Partial failure** | Some parts of a system broken while others work, often with no way to tell which. | 7, 8 |
| **Partial order** | Ordering where some pairs of items can't be compared (e.g., concurrent events). | 9 |
| **Participant** | Database node taking part in a two-phase commit. | 9 |
| **Partition-aware client** | Client that knows which node holds which partition and connects directly to the right one. | 6 |
| **Partition boundaries** | The key values where one partition's range ends and the next begins. | 6, 12 |
| **Partitioned hash join** | Map-side join when both inputs are partitioned the same way, so each mapper joins one matching pair of partitions. | 10 |
| **Partitioned log** | Log split into several partitions on different machines to scale throughput; order is kept only within each partition. | 11, 12 |
| **Partitioning proportionally to nodes** | Giving each node a fixed number of partitions, so the partition count grows with the cluster. | 6 |
| **Partition (shard, region, tablet, vnode, vBucket)** | One piece of a dataset that has been split across machines; the names differ by product. | 6, 9 |
| **Paxos** | The classic fault-tolerant consensus algorithm. | 9 |
| **Peer-to-peer network** | Network where participants talk directly to each other with no central server. | 8 |
| **Percolator** | Elasticsearch's feature for storing queries and matching each new document against them (stream search). | 11 |
| **Perfect failure detector** | Imaginary detector that always correctly knows whether a node has crashed; it can't exist on real networks. | 9 |
| **Pessimistic concurrency control** | Blocking (locking) whenever a conflict might happen, as two-phase locking does. | 7 |
| **Phantom** | Write in one transaction changing the result of a search query in another (e.g., inserting a row that matches its WHERE clause). | 7 |

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Phi Accrual failure detector** | Failure detector that adapts its timeout to observed network delays and reports a suspicion level. | 8 |
| **Physical clock** | Clock that measures real elapsed time (seconds), as opposed to a logical counter. | 8, 12 |
| **Pig** | A Hadoop tool with a scripting language for building MapReduce workflows. | 10 |
| **Pipelined execution** | Passing output from one operator to the next as soon as it's ready, rather than waiting for the whole stage. | 10 |
| **Point in time** | Specific date and time, as opposed to a duration. | 8 |
| **Polling** | Repeatedly asking "anything new?" instead of being notified. | 11, 12 |
| **Polyglot persistence** | Using several different kinds of database in one application, each for what it's good at. | 2 |
| **PostGIS** | A PostgreSQL extension for geographic data, using R-tree-like indexes. | 3 |
| **PostgreSQL** | Popular, feature-rich open-source relational database. | 5, 7 |
| **Postings list** | For one word, the list of IDs of documents that contain it. | 1, 3 |
| **ppm (parts per million)** | Unit for clock drift: 200 ppm means up to 17 seconds off per day. | 8 |
| **Predicate lock** | Lock on all records matching a condition, including ones that don't exist yet. | 7 |
| **Predictive analytics** | Using data to forecast things, including about individual people (credit risk, reoffending). | 12 |
| **Preemption** | Stopping a running task (or thread) to give its resources to a higher-priority one. | 8, 10 |
| **Premise** | Fact a transaction read and relied on when deciding what to write; if it changes before commit, SSI aborts. | 7 |
| **Pre-splitting** | Creating initial partitions in advance to avoid all writes hitting one partition at first. | 6 |
| **Priority queue** | Queue that always hands out the most important item next instead of the oldest. | 3, 7 |
| **Probabilistic algorithm** | Algorithm that gives approximate answers using much less memory (e.g., Bloom filters, HyperLogLog). | 11, 12 |
| **Procedural SQL (PL/SQL, T-SQL, PL/pgSQL)** | Each database vendor's programming language for writing stored procedures. | 7 |
| **Processing lag** | How far a stream processor's work is behind the newest events. | 11 |
| **Processing time** | When an event is processed by the stream processor, which can be much later than when it happened. | 11, 12 |
| **Producer** | Program that writes messages or events into a stream or queue. | 11 |
| **Property graph** | Graph model where vertices and edges each have a label and a set of key–value properties. | 2 |
| **Protocol Buffers (Protobuf)** | Google's binary encoding format, using a schema and numeric field tags. | 4 |
| **PTP (Precision Time Protocol)** | Protocol for very accurate clock synchronisation over a local network. | 8 |

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Publish/subscribe** | Producers publish messages to a topic, and everyone subscribed to it receives them. | 11, 12 |
| **Pure function** | Function whose output depends only on its input and that has no side effects. | 2, 12 |
| **Push notification** | Message the server sends to a device without the device asking first. | 11 |

## Q

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Quality of service (QoS)** | Network features that reserve capacity or prioritise some traffic. | 8 |
| **Quartz crystal oscillator** | The vibrating crystal that keeps time in a computer's hardware clock; it drifts with temperature. | 8 |
| **Query optimizer** | The part of a database that chooses how to execute a query (index use, join order...). | 2, 10 |
| **Queueing delay** | Time a request or packet spends waiting in a queue before being processed. | 1, 8 |
| **Queue / topic** | Named channel in a message broker that producers write to and consumers read from. | 4, 11 |
| **Quorum read** | Reading from a majority of replicas so the result reflects the latest committed write. | 5, 9 |

## R

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Race condition** | Bug where the result depends on the exact timing of concurrent operations. | 7, 11 |
| **Raft** | Widely used consensus algorithm designed to be easier to understand than Paxos (used by etcd). | 9 |
| **RAID** | Combining several disks so data survives the failure of one of them. | 1, 10 |
| **RAM** | Computer's main memory: fast, but its contents are lost on power loss. | 3, 10 |
| **Range scan / range query** | Fetching all records whose keys lie between two values. | 3, 6 |
| **RDBMS** | Relational database management system, such as PostgreSQL, MySQL or Oracle. | 2 |
| **RDD** | Resilient Distributed Dataset: Spark's abstraction that tracks how each piece of data was computed so it can be recomputed after failure. | 10 |
| **RDF** | Resource Description Framework: a graph model storing data as (subject, predicate, object) triples. | 2 |
| **RDMA** | Remote direct memory access: reading another machine's memory over the network with very low latency. | 8 |
| **Read-after-write (read-your-writes) consistency** | Guarantee that users always see their own submitted updates. | 5 |
| **Read committed** | Isolation level preventing dirty reads and dirty writes. | 7 |
| **Reader's schema / writer's schema** | In Avro, the schema data was written with versus the one the reading code expects; they are reconciled when decoding. | 4 |
| **Read-modify-write cycle** | Reading a value, changing it in application code and writing it back; prone to lost updates. | 7 |
| **Read path** | Work done lazily, only when someone asks a question. | 12 |
| **Read repair** | Client reading from several replicas notices a stale one and writes the newer value back to it. | 5, 9 |
| **Read-scaling architecture** | Adding many followers to serve more read traffic. | 5 |
| **Read skew / nonrepeatable read** | Seeing different parts of the database at different points in time within one transaction. | 7 |
| **Read uncommitted** | Weak isolation level that prevents dirty writes but allows dirty reads. | 7 |
| **Rebalancing** | Moving partitions between nodes so load stays even as the cluster changes. | 6 |
| **Recency guarantee** | Promise that a read returns the most recent value written; the essence of linearizability. | 9 |
| **Recursive common table expression** | SQL's WITH RECURSIVE syntax, which can express graph traversals. | 2 |
| **Red-black tree / AVL tree** | Self-balancing in-memory tree structures that keep keys sorted with O(log n) inserts and lookups; used for memtables. | 3 |
| **Redelivery** | Broker sending a message again because the consumer didn't acknowledge it. | 4, 11 |

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Redis** | Popular in-memory key-value store with data structures such as lists and sets. | 5, 7 |
| **Redshift / Vertica / Teradata** | Commercial column-oriented MPP data warehouse products. | 3 |
| **Reducer** | The function in MapReduce that receives all values for one key and produces output. | 10 |
| **Reduce-side join** | Join done in the reducers, after the shuffle brings records with the same key together. | 10 |
| **Refactoring** | Restructuring code to make it cleaner without changing what it does. | 1 |
| **Register** | In consistency theory, a single variable that can be read and written. | 9 |
| **Regular expression** | Compact pattern language for matching text. | 2, 4 |
| **Regular register** | Register where a read concurrent with a write may return either the old or the new value (weaker than linearizable). | 9 |
| **Relational algebra** | The mathematical foundation of SQL: operations like select, project and join on tables. | 2 |
| **Relational model** | Data organised as tables (relations) of rows, queried with SQL. | 2 |
| **Reliability** | System keeps working correctly (right function, acceptable performance) even when hardware, software or people make mistakes. | 1 |
| **Remote code execution** | Security hole that lets an attacker run their own code on your machine (e.g., via unsafe deserialization). | 4 |
| **Repeatable read** | Isolation level name with inconsistent meanings across databases; often it really means snapshot isolation. | 7 |
| **Replay** | Re-reading old messages from a log to reprocess them. | 11, 12 |
| **Replica** | Node holding a copy of the data. | 5, 9 |
| **Replication lag** | The delay between a write on the leader and it showing up on a follower. | 5, 9 |
| **Replication log** | The stream of changes a leader sends to its followers. | 5, 11 |
| **Replication topology** | The pattern of which leaders send writes to which (circular, star, all-to-all). | 5, 8 |
| **Reprocessing** | Re-running a (new) transformation over all old data to build a new derived view. | 11, 12 |
| **Request routing** | Getting each request to the node that holds the relevant partition. | Preface, 6 |
| **Response time** | What the client sees: total time from sending a request to receiving the answer. | 1, 8 |
| **REST / RESTful** | Style of web API built on HTTP, URLs and standard methods like GET and POST. | 4 |
| **RethinkDB** | Open-source document database that supports subscribing to query changes. | 2, 6 |
| **Retraction / correction (streams)** | Publishing an updated result for a window after late events arrive, replacing the earlier output. | 9, 11 |
| **Riak** | Open-source Dynamo-style leaderless key-value store. | 5, 6 |

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Ring buffer (circular buffer)** | Fixed-size buffer where new data overwrites the oldest once it is full. | 11 |
| **Rolling average** | Average over a moving window, such as the last five minutes. | 11 |
| **Rolling upgrade** | Updating servers one or a few at a time so the service stays up throughout. | 1, 4 |
| **Round-robin load balancer** | Load balancer that sends each new request to the next server in turn. | 6 |
| **Routing tier** | Layer of servers that forward each request to the node owning the relevant partition. | 6 |
| **Rowhammer** | Hardware effect where repeatedly accessing memory rows flips bits in neighbouring rows. | 12 |
| **Row-level lock** | Lock on a single row rather than on a whole table. | 7 |
| **Row-oriented storage** | Storing all values of each row together; good for transactional workloads. | 3 |
| **RPC (remote procedure call)** | Calling a function on another machine over the network as if it were local. | 4 |
| **RST / FIN packet** | TCP packets that end a connection; their arrival can signal that a remote process has died. | 8 |
| **RTOS** | Real-time operating system: one that guarantees CPU time to processes within deadlines. | 8 |
| **R-tree** | Index for multi-dimensional data such as geographic locations. | 3 |
| **Rule (Datalog)** | In Datalog, a named definition that derives new facts from existing facts (and from other rules, even recursively). | 2 |
| **Run-length encoding** | Compressing repeated values by storing "value × count" instead. | 3 |

## S

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Safety guarantees** | Promises a system makes about bad things never happening (e.g., no lost writes). | 7, 9 |
| **Safety property** | "Nothing bad happens" (e.g., no two nodes hold the same lock); once violated it can't be undone. | 8, 9 |
| **Samza** | Apache Samza: a stream processing framework closely tied to Kafka. | 11 |
| **SAP HANA** | SAP's in-memory, column-oriented database. | 3 |
| **Scalability** | System's ability to cope with increased load, usually by adding resources. | 1, 12 |
| **Scaling up / scaling out** | Scaling up (vertical) means a bigger machine; scaling out (horizontal) means more machines. | 1 |
| **Scatter/gather** | Sending a query to all partitions and combining the results. | 6 |
| **Schema evolution** | Changing a schema over time while keeping old and new data readable. | 4 |
| **Schemaless** | Databases that don't enforce a schema on write; really schema-on-read. | 2, 4 |
| **Schema migration** | Changing a database's schema (and possibly rewriting existing data). | 4, 12 |
| **Schema-on-read / schema-on-write** | Schema-on-read: structure is interpreted only when reading ("schemaless"). Schema-on-write: data is checked against a schema when stored. | 2, 4 |
| **Secondary sort** | Sorting the values within each reducer group in a particular order. | 10 |
| **Self-describing file** | File that includes its own schema, so any reader can decode it. | 4 |
| **Semantic web** | The idea of websites publishing machine-readable data (as RDF) alongside human-readable pages. | 2 |
| **Semi-synchronous replication** | One follower is synchronous and the rest asynchronous, so at least two nodes have every committed write. | 5 |
| **Sequence number** | Counter attached to each operation to order them. | 5, 9 |
| **Sequential consistency** | All nodes see operations in the same order, consistent with each client's program order, but not necessarily in real time. | 9 |
| **Serializable snapshot isolation (SSI)** | Optimistic algorithm giving full serializability on top of snapshot isolation by detecting dangerous conflicts at commit. | 7, 9 |
| **Server-sent events / WebSockets** | Protocols letting a server push messages to a browser over a long-lived connection. | 12 |
| **Service** | Program that offers an API over the network for other programs to use. | 4, 9 |
| **Service discovery** | Finding which IP address and port a service is currently running on. | 4, 9 |
| **Service time** | The time actually spent processing a request, excluding waiting in queues. | 1 |
| **Sessionization** | Grouping a user's events into separate visits (sessions). | 10, 12 |
| **Session timeout** | How long ZooKeeper waits without heartbeats before declaring a client's session dead. | 9 |
| **Sharding** | Another name for partitioning (see the table above): splitting data across machines. | Preface, 6 |
| **Shared-disk / shared-memory** | Scaling architectures where machines share one set of disks or one memory, as opposed to shared-nothing. | 4, 5 |

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Shredding** | Splitting a document-like structure into several relational tables. | 2 |
| **Shuffle** | The MapReduce step that sorts mapper output and sends all values for each key to the same reducer. | 10 |
| **Sibling pointers** | Links from each B-tree leaf page to its neighbours, so range scans can move along without going back to the root. | 3 |
| **Siblings** | Several concurrent values kept for one key until the application merges them. | 5 |
| **SIGSTOP / SIGCONT** | Unix signals that pause and resume a process; a paused process doesn't know time has passed. | 8 |
| **SIMD** | Single instruction, multiple data: CPU instructions that process several values at once. | 3 |
| **Simplicity** | Making a system easy to understand by removing accidental complexity. | 1 |
| **Single-object operation** | Atomic change to one record (e.g., increment or compare-and-set). | 7, 12 |
| **Size-tiered / leveled compaction** | Two strategies for scheduling LSM-tree merges: merge similar-sized files, or keep data in levels of fixed size. | 3 |
| **Slewing / smearing** | Adjusting a clock gradually (slightly faster or slower) instead of jumping, e.g., to spread a leap second over a day. | 8 |
| **Sloppy quorum** | Accepting writes on nodes outside a key's usual set when the usual ones are unreachable. | 5 |
| **SLO / SLA** | Service level objective / agreement: targets or contracts for performance and availability (e.g., p99 under 1 s). | 1 |
| **Slowly changing dimension (SCD)** | Keeping old versions of dimension data (e.g., a tax rate) with version IDs so joins stay reproducible. | 11 |
| **Snapshot** | Consistent copy of the data as it was at one point in time. | 7 |
| **Snapshot isolation** | Each transaction reads from a consistent snapshot of the database as of when it started. | 7 |
| **Snowflake schema** | Star schema where dimension tables are further split into sub-dimension tables. | 3 |
| **SOA** | Service-oriented architecture: building an application as a set of services; microservices are its modern form. | 4 |
| **SOAP** | An XML-based protocol for web service requests, described with WSDL. | 4 |
| **Solr** | Open-source search server built on Lucene. | 1, 10 |
| **Sort key** | The column(s) a column store sorts its rows by, which speeds up range queries and compression. | 3 |
| **Sort-merge join** | Joining two datasets by sorting both by key and walking through them together. | 10 |
| **Space-filling curve** | Way to map two-dimensional locations to one-dimensional keys so a normal index can be used. | 3 |
| **Spanner** | Google's globally distributed SQL database, which uses TrueTime clocks to order transactions. | 8 |
| **Spark** | Apache Spark: a popular distributed engine for batch (and microbatch streaming) processing. | 10 |
| **SPARQL** | Query language for RDF triple-stores. | 2 |

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Speculative execution** | Starting a duplicate copy of a slow task on another machine and using whichever finishes first. | 10 |
| **SQL** | Structured Query Language: the standard declarative language for relational databases. | 2, 10 |
| **SQL injection / cross-site scripting** | Common web attacks where untrusted input is run as database commands or as script in other users' browsers. | 8 |
| **SQL Server** | Microsoft's commercial relational database. | 3, 7 |
| **SSD** | Solid-state drive: flash-memory storage, much faster than spinning hard disks. | 3, 7 |
| **SSTable** | Sorted String Table: an immutable file of key–value pairs sorted by key. | 3 |
| **Stable storage** | Storage that survives crashes (e.g., disk), assumed by the crash-recovery model. | 8 |
| **Star schema** | Data warehouse layout with one big fact table referencing smaller dimension tables. | 3 |
| **Stateless / stateful service** | Stateless server forgets everything between requests (state lives in a database); a stateful one keeps data itself. | 1, 12 |
| **State machine replication** | Every replica applies the same deterministic commands in the same order, so they all end in the same state. | 9, 11 |
| **Statement-based replication** | Replicating by sending each SQL statement to followers; breaks with nondeterministic functions like NOW(). | 5 |
| **Static vs dynamic type checking** | Checking types before the program runs (static) versus while it runs (dynamic); compared to schema-on-write versus schema-on-read. | 2, 4 |
| **StatsD / Brubeck** | Metrics daemons that receive counters over UDP and aggregate them. | 11 |
| **stdin / stdout** | Program's standard input and output streams, which Unix pipes connect together. | 10 |
| **Steal time** | CPU time a virtual machine loses to other VMs on the same physical host. | 8 |
| **Stop-the-world GC pause** | Pause where garbage collection halts all program threads. | 8 |
| **Storage engine** | The part of a database that decides how data is laid out on disk and in memory. | 3, 5 |
| **Store buffer** | Per-CPU-core queue of pending memory writes, one reason multi-core memory isn't linearizable. | 9 |
| **Storm** | Apache Storm: an early open-source stream processing framework. | 11, 12 |
| **Straggler** | Task that runs much slower than the rest, or an event that arrives late. | 10, 11 |
| **Stream** | Data that arrives gradually over time and never really ends. | 11, 12 |
| **Stream analytics** | Computing aggregates and statistics over a stream, such as rates or averages per time window. | 11 |
| **Stream-stream join (window join)** | Matching events from two streams that occur close together in time. | 11 |
| **Stream-table join (enrichment)** | Adding information to each event by looking it up in a table, often a local copy kept up to date via CDC. | 11, 12 |
| **Strict quorum** | Quorum where reads and writes overlap (w + r > n), so a read normally sees the latest write. | 5, 9 |
| **Strict serializability** | Serializability plus linearizability: transactions appear in an order consistent with real time. | 9 |

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Supermajority** | More than two-thirds of the nodes; needed by Byzantine fault-tolerant protocols. | 8 |
| **Surveillance** | Tracking people mainly to serve someone else's interests (e.g., advertisers). | 12 |
| **Sushi principle** | "Raw data is better": store data in its original form and interpret it later. | 10 |
| **Synchronous network** | Network with reserved bandwidth and bounded delays, like old telephone circuits. | 8, 12 |
| **Systematic error** | Fault that hits many nodes at once (e.g., a software bug triggered by the same input). | 1 |
| **System model** | The set of assumptions about what can go wrong (crashes, delays, clock errors) that an algorithm is designed for. | 8 |
| **System R** | IBM's 1970s research database that pioneered SQL and many transaction ideas. | 7 |
| **Systems thinking** | Considering the whole system, including the humans in it, to foresee consequences. | 12 |

## T

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Tableau** | Popular commercial tool for visual data analysis and dashboards. | 10 |
| **Table-table join** | Keeping the join of two changing tables up to date by consuming both change streams. | 11 |
| **tail -f** | A Unix command that prints new lines as they are appended to a file — like a consumer following a log. | 11 |
| **Tail latency** | The response times of the slowest requests (high percentiles like p99). | 1, 3 |
| **Tail latency amplification** | When one user request needs many backend calls, it's slow if any of them is slow. | 1, 6 |
| **Tandem NonStop SQL** | A 1980s fault-tolerant commercial database, an early example of partitioning with transactions. | 6, 10 |
| **TCP** | Transmission Control Protocol: builds a reliable, ordered byte stream on top of IP, with retransmission and flow control. | 8, 12 |
| **t-digest / forward decay / HdrHistogram** | Algorithms and libraries for computing approximate percentiles cheaply over a stream of measurements. | 1 |
| **Telemetry** | Detailed monitoring data (metrics, error rates) sent from running systems. | 1 |
| **Term dictionary** | In a full-text index, the sorted list of all words, each pointing to its postings list. | 3, 10 |
| **Term-partitioned (global) index** | Secondary index split up by the indexed value, so a read hits one partition but a write may touch several. | 6 |
| **Test-driven development (TDD)** | Writing automated tests before the code they test. | 1 |
| **Textual format** | Human-readable encodings such as JSON, XML and CSV. | 4 |
| **Tez** | Dataflow engine for Hadoop that runs workflows without writing every step to HDFS. | 10 |
| **Thinking like a vertex** | The Pregel style of writing graph algorithms as code that runs per vertex and talks to neighbours by messages. | 10 |
| **Thrashing** | Computer spending most of its time swapping memory to disk, doing little real work. | 8 |
| **Three-way merge** | Merging two changed versions by comparing each against their common ancestor, as Git does. | 5 |
| **Thrift** | Binary encoding and RPC framework from Facebook, using schemas and numeric field tags. | 4 |
| **Throughput** | How much work a system handles per unit of time (e.g., records per second). | 1, 10 |
| **Timeliness** | Users see an up-to-date state; violations are temporary. | 12 |
| **Time-of-day clock** | Clock giving the current date and time; it can jump backwards when corrected. | 8 |
| **TLS/SSL** | The encryption protocol that secures HTTPS and many other network connections. | 4, 12 |
| **Tombstone** | Special marker recording that a key was deleted, so the deletion survives merging and replication. | 3, 5 |

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Top-of-rack switch / router** | Network devices: a top-of-rack switch connects the servers in one rack; a router forwards packets between networks. | 8 |
| **Total order broadcast (atomic broadcast)** | Delivering every message reliably to all nodes in the same order; equivalent to consensus. | 9 |
| **Transaction ID (txid)** | Number the database gives each transaction, used to tag the row versions it creates. | 7, 9 |
| **Transaction processing** | Handling many small reads and writes as business events happen (sales, orders, payments). | 2, 3 |
| **Transitive closure** | Finding everything reachable from a starting point through any number of steps. | 10 |
| **Trie** | Tree that stores strings letter by letter, so words with the same prefix share a path. | 3 |
| **Trigger** | Code a database runs automatically when data changes. | 5, 11 |
| **Trigger-based replication** | Using database triggers to copy changes into a separate table that an external process replicates; flexible but slower. | 5, 11 |
| **Triple-store** | Graph database storing (subject, predicate, object) triples, such as RDF data. | 2 |
| **TrueTime** | Google's clock API in Spanner that returns a time range guaranteed to contain the true time. | 8 |
| **Tumbling / hopping / sliding / session window** | Ways to group stream events by time: fixed non-overlapping blocks; fixed overlapping blocks; a moving interval; or bursts of activity separated by gaps. | 11 |
| **Tungsten Replicator / BDR / Bucardo** | Add-on tools providing multi-leader replication for MySQL (Tungsten) and PostgreSQL (BDR, Bucardo). | 5 |
| **Tuple** | In the relational model, the formal name for a row. | 2, 10 |
| **Turtle / N3** | Human-readable text formats for writing RDF triples. | 2 |
| **Twitter Snowflake** | Twitter's service that hands out unique, roughly time-ordered 64-bit IDs. | 8 |
| **Two Generals Problem** | The puzzle showing two parties can never be certain they agree over an unreliable channel. | 8 |

## U

| Term | Plain-English meaning | Ch |
|---|---|---|
| **UDP** | User Datagram Protocol: sends packets with no retransmission or ordering; used where late data is useless (e.g., voice calls). | 8, 11 |
| **UDP multicast** | Sending one UDP packet to many receivers at once; low latency but unreliable. | 9, 11 |
| **Unbundling databases** | Treating the features inside a database (indexes, views, replication) as separate components wired together with logs. | 12 |
| **Unicode / UTF-8** | Unicode is the universal character set; UTF-8 is its most common byte encoding. | 4 |
| **Unified batch and stream processing** | Doing both historical reprocessing and live processing in one engine with the same code. | 12 |
| **Uniform consensus** | Consensus where even nodes that later crash must not have decided differently. | 9 |
| **Uniform interface** | Everything speaking the same format (in Unix, a stream of bytes, often lines of text), so any tool can connect to any other. | 10 |
| **Union type** | Schema type allowing a field to be one of several types (e.g., null or string) in Avro. | 4 |
| **Uniqueness constraint** | Rule that no two records may share a value (a username, a seat, a request ID). | 9, 12 |
| **Unix pipe** | Connecting one program's output to the next program's input (the vertical-bar character in a shell command). | 10 |
| **URI** | Uniform Resource Identifier: a string that names a resource; URLs are a kind of URI. | 4 |
| **URL** | Uniform Resource Locator: a web address. | 10 |
| **UUID** | Universally unique identifier: a 128-bit random ID that is practically guaranteed to be unique. | 5, 12 |

## V

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Vacuum** | PostgreSQL's background process that removes old row versions no transaction can see any more. | 7 |
| **Variable-length integer (varint)** | Encoding small numbers in fewer bytes than large ones. | 4 |
| **Vectorized processing** | Processing a batch of column values in a tight loop at once, using CPU caches and SIMD efficiently. | 3, 10 |
| **Version vector** | Set of counters, one per replica, used to tell whether two versions are concurrent or one supersedes the other. | 5, 9 |
| **Vertex** | Node (thing) in a graph, joined to others by edges. | 2, 10 |
| **Viewstamped Replication (VSR)** | Consensus algorithm from the 1980s, similar in spirit to Raft. | 9 |
| **Virtual machine (VM)** | Simulated computer running on shared physical hardware. | 1, 8 |
| **Virtual view** | Saved query that is re-run whenever it is read (an ordinary SQL view). | 3 |
| **VoIP** | Voice over IP: phone calls carried over the internet. | 8 |
| **Voldemort** | LinkedIn's open-source Dynamo-style key-value store. | 5, 6 |
| **VoltDB** | In-memory database that runs transactions one at a time on a single thread per partition. | 7, 11 |

## W

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Wall-clock time** | The current date and time of day, as shown by a time-of-day clock. | 8 |
| **Watch / change notification** | A ZooKeeper feature letting a client be notified when some data changes. | 9, 12 |
| **Webhook** | A URL that one service calls when an event happens, to notify another service. | 11 |
| **Web service** | Service accessed using HTTP. | 4, 9 |
| **Window** | Time interval used to group stream events for aggregation or joining. | 11 |
| **Workflow** | Chain of batch jobs where each one's output becomes the next one's input. | 10 |
| **Workflow scheduler** | Tool (e.g., Oozie, Azkaban, Airflow) that runs batch jobs in dependency order. | 10 |
| **Working set** | The data a computation needs in memory at once. | 10, 11 |
| **Write-ahead log (WAL)** | Append-only file where every change is written before it's applied, so a crash can be recovered from. | 3, 5 |
| **Write amplification** | One logical write causing several physical writes to disk over time. | 3 |
| **Write conflict** | Two concurrent writes changing the same data in incompatible ways. | 5, 12 |
| **Write path** | Work done eagerly when data arrives (updating indexes, caches and views). | 12 |
| **Write skew** | Two transactions read the same data, then update different records, together breaking a rule each one checked. | 7 |
| **WSDL** | Web Services Description Language: an XML format describing a SOAP API. | 4 |

## X

| Term | Plain-English meaning | Ch |
|---|---|---|
| **XA** | X/Open eXtended Architecture: the standard API for two-phase commit across heterogeneous systems. | 9 |
| **XML** | Extensible Markup Language: a verbose text format with tags, used for documents and data. | 2, 4 |
| **XPath / XSL** | Languages for selecting parts of an XML document (XPath) and transforming them (XSL). | 2 |

## Y

| Term | Plain-English meaning | Ch |
|---|---|---|
| **YARN** | Hadoop's cluster resource manager, which schedules jobs onto machines. | 9, 10 |

## Z

| Term | Plain-English meaning | Ch |
|---|---|---|
| **Zab** | ZooKeeper's consensus (total order broadcast) protocol. | 9 |
| **ZeroMQ / nanomsg** | Libraries for sending messages directly between programs without a broker. | 11 |
| **ZooKeeper** | Open-source coordination service for leader election, locks, configuration and service discovery. | 6, 9 |
| **zxid** | ZooKeeper's transaction ID, which increases with each change and can serve as a fencing token. | 8, 9 |